

C5 - MULTIMODAL RECOGNITION

Task 5: Diffusion Models (I)

Team 1:

Diego Hernández Antón · Oriol Juan Sabater · Valentin Micu Hontan
Xavier Pacheco Bach · Benet Ramió Comas



MASTER IN
COMPUTER VISION
Barcelona



Setup & Introduction

Diffusion

Process of generating **high-fidelity visual images** that represent the content of a textual prompt.



It integrates **Natural Language Processing (NLP)** to automatically analyze textual data and **generative diffusion models** to produce coherent, contextually relevant images that reflect the objects, actions, and scenes described in the text.

Goals

- **Primary Goal:** Use **Latent Diffusion Models** to generate targeted synthetic data and improve a flawed image captioning model.
- **Task 1(Diffusion Exploration):** Deploy Stable Diffusion models and experiment with **inference parameters** like schedulers, CFG strength and denoising steps.
- **Task 2 (Targeted Generation):** Identify a specific **bias** or **limitation** in our existing captioning model, and use LLMs to help generate a targeted synthetic dataset to fix it.
- **Task 3 (Fine-Tuning & Evaluation):** **Retrain** our captioning model using the new synthetic data and critically analyze whether the targeted augmentation solved our problem.

Frameworks & Models

Hugging Face

- **Frameworks:** [Stability AI](#), [Runwayml](#)
- **Diffusion Models:** Stable-diffusion - v1-5, xl-base 1.0, xl-turbo
- **Captioning model:** Qwen



Dffusers



- *We first outline our experimental framework before detailing task results.*
- **Github Repository:** <https://github.com/valy3124/C5-Team1>

Dataset: VizWiz

Train

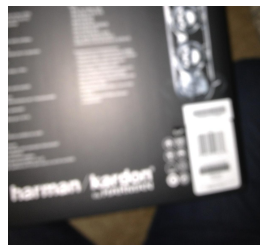
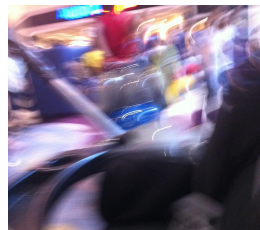
Training images: 23,431
Training captions: 117,155

Test

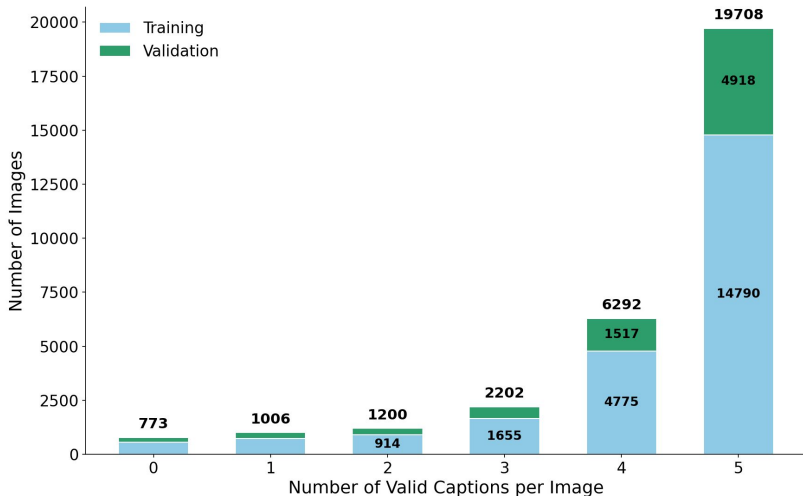
We used the official [validation set](#).

Test images: 7,542
Test captions: 33,186

Note: These images were taken by blind persons. Because the photographers couldn't see the viewfinder, the images often suffer from poor lighting, extreme blur, "finger-over-the-lens," or the subject being entirely out of frame.



Distribution: Captions per Image(Full Data Split)



Key Observations

- There are some images with **no caption** that had to be removed.
- As per project requirement, we do **not use** the **test set**.
- There is a significant **imbalance** in the number of **captions per image**, which may affect model training.

Note: This slide is based on the dataset slide from previous week

Metrics Description

Note: From previous week

We use **METEOR** as our main metric during evaluation because it focuses on **recall** and handles **synonyms** (using a lexical database). This means it checks if the model actually gets the **meaning** of the image right, rather than just demanding exact word matches like BLEU does. We save **CIDEr** strictly for the final comparison between the baseline and our final model. Since CIDEr uses **TF-IDF** to give more weight to specific objects and less to common filler words, it's the most direct way to show whether our architecture changes actually **helped the model** recognize the important visual details in the scene.

*

METEOR

Def: Harmonic mean of precision and recall (favoring recall), with a penalty for fragmentation.

$$F_{mean} \cdot (1 - \text{Pen})$$

where

Why: Rewards capturing the actual meaning and heavily penalizes scrambled word order.

$$F_{mean} = \frac{P \cdot R}{\alpha P + (1 - \alpha) R},$$

$$\text{Pen} = \gamma \left(\frac{c}{m} \right)^\beta$$

BLEU

Def: Measures exact n-gram overlap with references (1 for words, 2 for pairs).

BLEU 1

$$\min \left(1, \frac{\text{output-length}}{\text{reference-length}} \right) \text{precision}_1$$

Why: Verifies exact vocabulary match and penalizes artificially short sentences.

BLEU 2

$$\min \left(1, \frac{\text{output-length}}{\text{reference-length}} \right) \left(\prod_{i=1}^2 \text{precision}_i \right)^{\frac{1}{2}}$$

CIDEr

Def: Measures consensus with human references by weighting specific, informative words using TF-IDF.

$$\frac{1}{m} \sum_{j=1}^m \frac{\vec{g}^n(c_i) \cdot \vec{g}^n(s_{ij})}{\|\vec{g}^n(c_i)\| \|\vec{g}^n(s_{ij})\|}$$

where

Why: Ensures the model identifies key visual objects instead of just repeating common filler words.

- m : No. of human references.
- c_i : Candidate caption.
- s_{ij} : Reference caption j .
- $\vec{g}^n$: TF-IDF weights.

ROUGE-L

Def: Measures the longest common subsequence (LCS) between the generated and reference text.

$$2 \cdot P \cdot RP + R$$

where

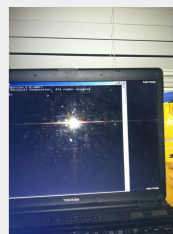
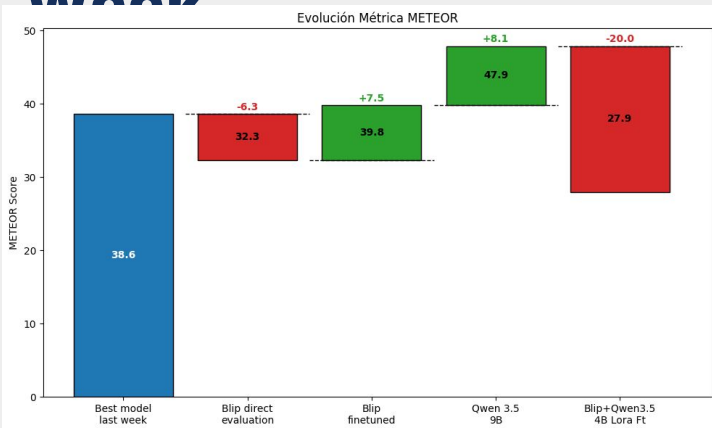
$$P = \frac{LCS}{c}, R = \frac{LCS}{r}$$

Why: Ensures the correct order of visual events, even if words aren't perfectly consecutive.

LCS: Length of the Longest Common Subsequence.
c, r: Total words in candidate (c) and reference (r).

*

Summary Previous Week



Caption - Best LW- **BAD**

a laptop screen is on top of a laptop

Caption - BLIP FT - **BETTER**

a laptop computer with a blue screen and a yellow box in the background

Caption - Qwen3.5 - **BETTER**

A Toshiba laptop displaying a Microsoft command prompt with "Safe Mode" visible on the screen.



Caption - Qwen3.5 0.8B

The image displays a close-up view of a person's face, specifically focusing on the right side of the face. The lighting is bright, casting distinct

Caption - Qwen3.5 4B - **BAD**

The image displays a close-up of a dark glass bottle, likely containing wine or beer, set against a blurred background. The bottle features a label with



Conclusions

- **BLIP** is the best pre-trained image captioning model because it achieves the highest scores across all metrics, benefiting from its vision-language **pre-training specifically designed for captioning tasks.** (Task 1)
- **Fine-tuning both encoder and decoder in BLIP** is the best freezing strategy because it **improves alignment between visual features and generated text.** (Task 1)
- A direct evaluation with **Qwen3.5 9B** obtains the highest METEOR since **its superior vocabulary breadth excels at semantic recall and synonym matching.** (Task 2)
- The **LoRA fine-tuning of Qwen3.5 4B** suffers a **drop in performance, not directly comparable with direct evaluation** since here we are using a **frozen encoder** (best in Task 1), a **linear projector**, and for the decoder the **Qwen model**. The **linear projector fails** to bridge the modality gap between BLIP visual and Qwen text embeddings, leading to **modality collapse.** (Task 2)

Task a) Models - SD v1.5 (*)

Note: all the models shown in the next 3 slides are latent models from the same Stable Diffusion family.

Core specs

- **Release:** Late 2022 by Runway & Stability AI.
- **Resolution:** Trained natively on 512x512 pixel images.
- **Text Encoder:** Uses a single CLIP ViT-L/14 encoder.
- **Size:** Relatively lightweight (~860M parameters), making it incredibly efficient to run.

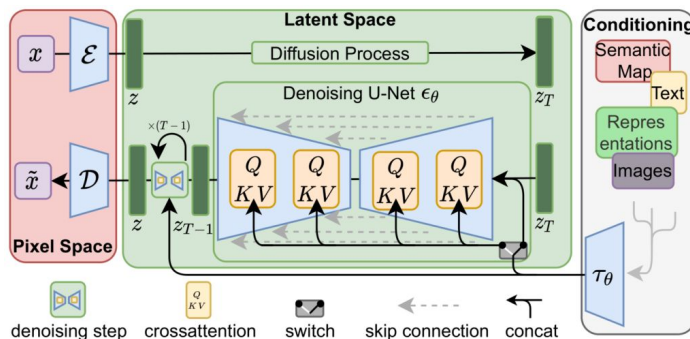
Why Use SD 1.5?

- **Hardware friendly:** Runs on consumer GPUs with low VRAM (e.g., 4GB-8GB). We use RTX 3090 with 24GB
- **Massive Ecosystem:** Has the largest community library of fine-tunes, LoRAs, and ControlNets.
- **Speed:** Generates images much faster than its XL successors.

Limitations

- **Resolution Limit:** Images larger than 512x512 can suffer from duplicated subjects (e.g., two heads) without special upscaling techniques.
- **Prompt Adherence:** Struggles with complex, multi-subject prompts compared to newer models.
- **Text Generation:** Cannot reliably generate legible text within images.

Model Architecture



Task a) Models - SDXL Base 1.0 (*)

Core specs

- **Release:** Mid-2023 by Stability AI.
- **Resolution:** Trained natively for **1024x1024** im.
- **Dual Encoders:** Uses **two text encoders** instead of one, drastically improving its ability to understand complex prompts.
- **Larger Latents:** Operates in a **larger** 128x128 latent space (as shown in your diagram), capturing much finer details.

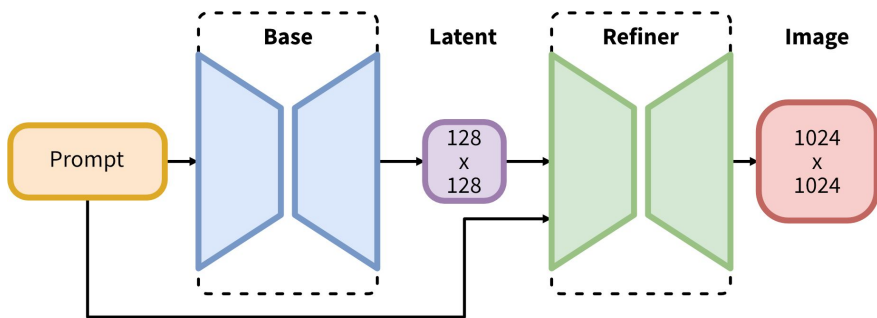
Why SDXL?

- **Massive Quality Jump:** Overwhelmingly preferred by users* for composition and realism compared to SD 1.5 and 2.1 (see the Win Rate chart).
- **Prompt Adherence:** Actually follows complex instructions (e.g., placing specific objects in specific background locations).

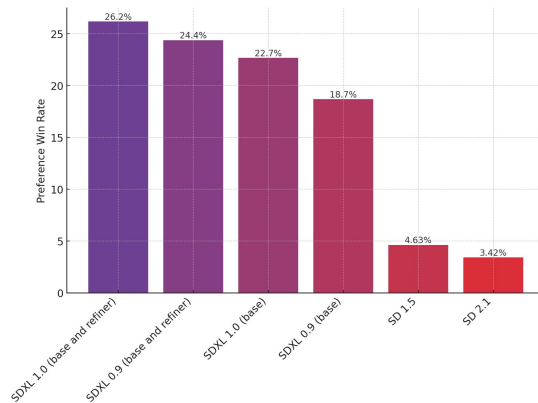
Resource Demands

- **Two-Step Process:** Introduces an optional **Refiner** model. The "Base" generates the core latents, and the "Refiner" polishes background details.
- **Hardware Heavy:** Because the UNet is 3x larger than SD 1.5, it takes longer to generate each image.

Model Architecture



User preference for SDXL published by StabilityAI



*<https://huggingface.co/stabilityai/stable-diffusion-xl-base-1.0>

Task a) Models - SDXL Turbo (*)

ADD Architecture

- **Distillation:** Released in late **2023**, this is a **distilled** version of SDXL 1.0.
- **ADD Technique:** Uses **Adversarial Diffusion Distillation** (as shown in the diagram).
- **Hybrid Approach:** It combines learning from a "Teacher" model with a Discriminator (adversarial loss, similar to GANs) to ensure the output looks like a real image immediately.

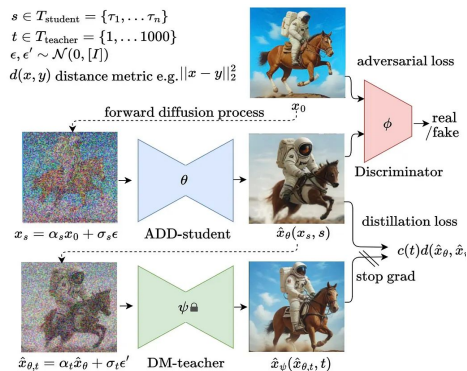
Real-Time Generation

- **Step Reduction:** While SD 1.5/SDXL typically require 20 to 50 denoising steps, SDXL Turbo is designed to generate images in just **1 to 4 steps**.
- **Real-time:** This reduction in steps allows for near **real-time generation**, where the image updates instantly.

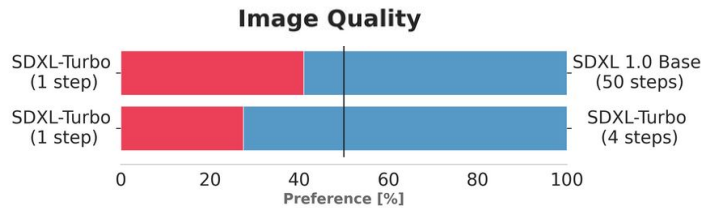
Quality vs. Steps

- **Impressive 1-Step:** As seen in our chart*, a **1-step Turbo** image is highly competitive, nearly matching the user preference of a **50-step SDXL Base** image.
- **Trade-off:** It sacrifices a bit of the **ultra-high resolution** detail of SDXL Base in exchange for **speed**.

Models Architecture



*User preference for SDXL published by StabilityAI



Task b) Parameter Exploration

For our **parameter exploration**, we selected three diverse ground-truth **reference images** (outdoor, indoor, and close-up) alongside their captions to evaluate different generation setups. Although we ran experiments on all three images across all the models, the upcoming slides will **not display** every image for every test. Instead, to keep the presentation clear and concise, we highlight only the **single most illustrative** image for each parameter tweak, specifically focusing on results generated by the **SDXL model** because provides the highest baseline quality. Throughout this exploration, we specifically **tested** changes in prompt style, schedulers, number of denoising steps, CFG scale, and negative prompting.

GT Captions

- A house has a car in the driveway and it is sunny outside.
- A white and tan two story house with a tree in the front yard under a blue sky.
- The front of two houses with four antenna towers in the background.
- A two story house with a tree in the yard.
- A landscape of a neighborhood with a two story house and a tree right in front of it along with a driveway.



GT Captions

- An office room has a TV and a table with papers and a bottle of liquid on it.
- An office or classroom with a TV on a metal AV stand is in the background of a white table with red chair and glass beverage bottle.
- A frosted white and orange glass bottle standing on a table with a glass and bowl.
- A flat screen TV is sitting on a utility cart near a table.



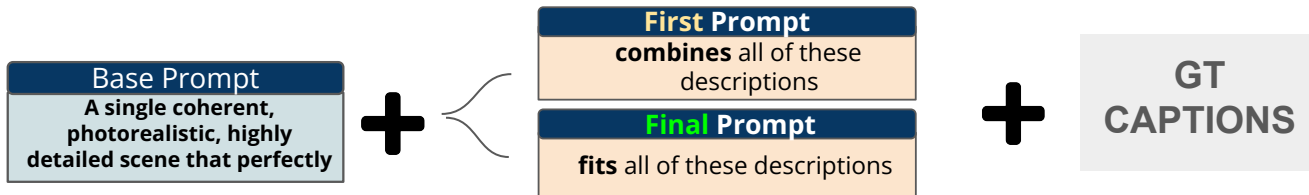
GT Captions

- A Hulk toy action figure on a carpet area.
- Green action figure of the Incredible Hulk lying on a brown carpet.
- Green plastic hulk toy that has purple pants on a carpeted floor.
- A toy of the Hulk wearing purple pants is on the carpet.
- An incredible hulk action figure laying on brown carpet.



Task b) Exploration: Prompt

Subsection Motivation: We first played around with **different prompts** to see how they behave. We tried feeding **all GT captions** simultaneously. Asking the model to 'combine' them forced it to treat the descriptions as a rigid checklist, creating an **unnatural scene**. By asking for a scene that 'fits' the descriptions instead, we encouraged the model to use them as cohesive constraints for a natural, **unified environment**.



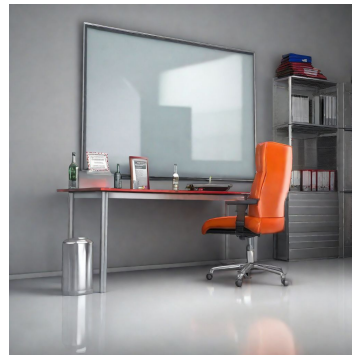
Results

- **First Prompt ("combines"):** Unnatural and visually weird. It **forces elements together** literally (e.g., placing paper textures on the TV and the bottle on the floor).
- **Final Prompt ("fits"):** Significantly **more coherent** and realistic. Elements are placed logically within a natural office space.
- **Concept Bleeding:** Both attempts struggle with **attribute binding** across multiple sentences. We see that the model incorrectly takes "orange" from the bottle description and applies it to the "red" chair.

First Prompt



Final Prompt



Task b) Exploration: Scheduler

Note: x_t : noisy image | ϵ_{θ} : predicted noise | α : schedule | $\bar{\alpha}$: cumulative | σ, z : randomness | $\Delta t, h$: step size

Subsection Motivation: The scheduler (or sampler) dictates the mathematical process of iteratively removing noise to form the image. We tested various schedulers using the exact same prompt, and step count to observe their impact on image coherence, detail retention, and how strictly they follow the conditioning.

DDIM

Def: A non-Markovian, deterministic solver. It skips steps to generate images much faster than DDPM.

$$x_{t-1} = \sqrt{\alpha_{t-1}/\alpha_t}(x_t - \sqrt{1-\alpha_t}\epsilon_{\theta}) + \sqrt{1-\alpha_{t-1}}\epsilon_{\theta}$$

DDPM

Def: The Markovian solver. It relies on a stochastic process, adding random noise at each step.

$$x_{t-1} = \alpha_t^{-1/2}(x_t - (1-\alpha_t)(1-\bar{\alpha}_t)^{-1/2}\epsilon_{\theta}) + \sigma_t z$$

Euler

Def: The simplest 1st-order ODE solver. Fast, but less detailed.

$$x_{t-1} = x_t - \Delta t \epsilon_{\theta}$$

DPM++

Def: Advanced, higher-order ODE solver. Achieves excellent detail in very few steps.

$$x_{t-1} = e^{-h}x_t + (1 - e^{-h})\epsilon_{\theta}$$

Results

- **DDIM & Euler:** We've seen that it generates nearly identical, coherent rooms, highlighting the predictable stability of **deterministic solvers**.
- **DDPM:** Diverged completely due to **injected random noise** (stochastic process). It hallucinated a brighter room and **bizarre** object placements.
- **DPM++:** Maintained the stable composition of DDIM/Euler but distinctly refined the fine details (e.g., adding a monitor), showing its **higher-order** capabilities.

CHOSEN

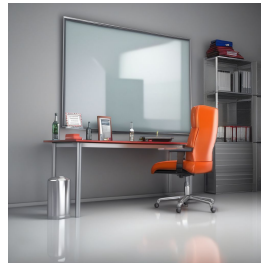
DDIM



DDPM



Euler



DPM++



Task b) Exploration: CFG

Subsection Motivation: The Classifier-Free Guidance (CFG) scale determines how strictly the generation process **adheres** to the text prompt. Lower values allow the model more **creative freedom**, while higher values force it to follow the text literally. We explored this parameter to find the exact threshold where the model perfectly obeys our caption without degrading the underlying image quality.

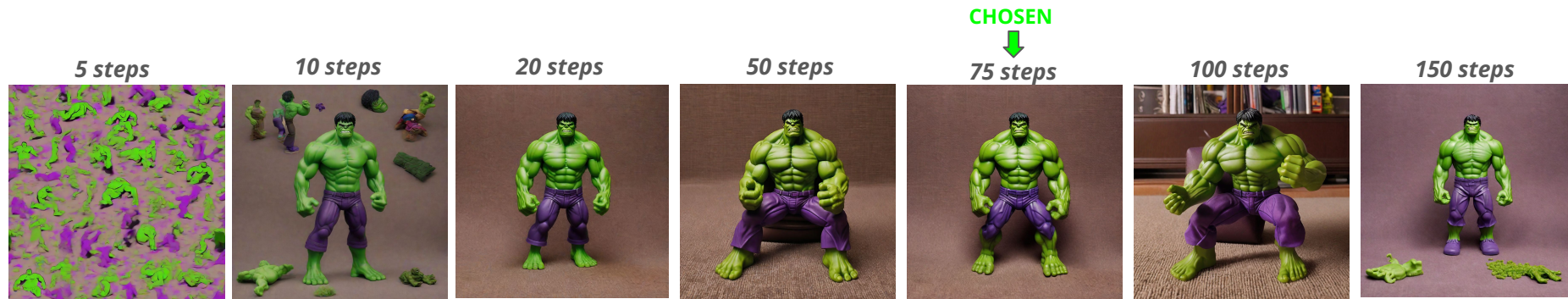


Results

- **Low CFG (1 - 2):** The model largely **ignores** the prompt. At CFG 1, it generates a disjointed, chaotic texture of distant buildings. By CFG 2, a main structure emerges, but it is hazy, washed out, and lacks definition.
- **CFG 5 (Transition):** The prompt is followed, but the image remains slightly **muted** and lacks crisp contrast.
- **CFG 7.5 (Chosen):** Perfect equilibrium. The model strictly **follows** the prompt (generating the distinct houses) while maintaining natural lighting, soft shadows, and the realism required by our dataset.
- **High CFG (15 - 25):** The model is forced to over-index on the prompt, "burning" the image. Colors become hyper-saturated, contrast is aggressively harsh, and the houses start looking **cartoonish** and like artificial 3D renders, completely losing real-world realism.

Task b) Exploration: Denoise Steps

Subsection Motivation: The number of denoising steps tells us how many iterations the model uses to refine the image from pure random noise. We explored this parameter to identify the optimal 'sweet spot'—balancing computational efficiency (generation time) with visual quality, and observing when the model hits diminishing returns.



Results

- **Under 20 Steps:** The image is **under-processed**. It starts as a pattern of noise at 5 steps, and develops severe structural artifacts, like disjointed, floating limbs by 10 steps.
- **20 - 50 Steps:** The main subject successfully converges and becomes coherent, though the background and lighting are **still shifting** and stabilizing.
- **75 Steps (Chosen):** This hits the **optimal sweet spot**. It delivers sharp details, realistic toy textures, and a clean composition without wasting unnecessary compute time.
- **100+ Steps (Diminishing Returns):** Pushing steps too high causes over-processing. The background unexpectedly clutters at 100 steps, and **visual artifacts** reappear on the floor at 150 steps.

Task b) Exploration: Neg Prompting

Note: Aesthetic selection is subjective, but "Light" offered the best balance here. Later in the project, we implemented a more robust, dataset-specific negative prompting technique.

Subsection Motivation: Negative prompting allows us to **instruct** the model on what not to generate (e.g., 'ugly', 'blurry'). We explored varying intensities of negative constraints, from **nothing** to a **heavy list** of restrictive terms (very similar to the dataset images), to observe how actively filtering out 'bad' traits affects the image's overall composition, lighting, and adherence to our ground-truth captions.



Results

- **Without Neg Prompting:** The overall composition feels **aesthetically flat** and washed out. Most importantly, it completely misses key elements from the captions, rendering a blank whiteboard instead of the requested television.
- **Light Neg Prompting (Chosen):** This significantly **improves** the lighting, contrast, and realism of the room. It successfully forces the model to generate the "TV" on the desk, matching the GT descriptions.
- **Heavy Neg Prompting:** Over-constraining the model with too many negative terms triggers severe **visual hallucinations**. While the room is brighter, it introduces bizarre, unintended artifacts, such as the massive metallic cylinder placed awkwardly in the foreground.

Task b) Trade-off: Time/Quality

Time vs. Quality

Evaluating these models reveals a stark trade-off between how long an image takes to generate and how good it looks:

- **SD v1.5 (Baseline)**: Takes **34s** per image. Produces **decent results** but struggles slightly with **structural artifacts** and **background clutter**.
- **SDXL 1.0 Base**: Takes **151s** per image. Delivers a massive leap in **quality**, yielding highly **realistic** and well-composed images, but at an **unfeasible computational cost**.
- **SDXL Turbo**: Takes just **1.2s** per image. Disrupts the trade-off entirely, offering **near real-time generation** while achieving a visual quality that **outperforms SD 1.5** and **rivals** the heavy **SDXL** base model.

Task D Decision: SDXL Turbo

- **Scale & Deadline**: We need **~5,000 synthetic images** (20% of the dataset) in **under a week**. SDXL Base is too slow; Turbo makes this **data augmentation feasible**.
- **Hyperparameters**: We selected two fast inference setups to balance **speed and quality**: 2S0CFG (2 steps, 0 CFG) and 4S1CFG (4 steps, 1.0 CFG).
- **Domain Match**: VizWiz photos are inherently **blurry and noisy** (taken by blind photographers). Turbo's slightly **lower fidelity** naturally mimics these **real-world flaws** better than the "perfect" SDXL Base.

Model	Generating time/image
SD v1.5	34s
SDXL 1.0 base	151s
SDXL Turbo	1.2s

SD 1.5



SD XL



SD XL Turbo



Task c) Task Definition & Model Selection

Section Motivation: In this section, we first introduce the main task of this week's project and justify our choice of captioning model to work with. In the following slides we will walk through our reasoning to define a clear problem statement and formulate a research question with an improvement goal.

Definition of the Task

Generate synthetic text-image pairs to improve our captioning model from previous week.

Possible Directions:

- Addressing dataset imbalance.
- Mitigating bias through more diverse samples.
- Generic data augmentation (broader, balanced training set).

Why it can go wrong:

- May introduce noise and artifacts.
- Distribution shift from real-world data (covariate shift).
- Can reinforce existing biases if not controlled.
- Costly to generate and filter quality samples

Selected Model

Among all the models explored in the previous week, we selected our **fully fine-tuned BLIP (*) model** for further analysis for two key reasons:

- Although it did not achieve the highest metrics overall, it consistently **outperformed all models from week 3** and **produced reasonably good results**.
- The top-performing model (**Qwen 3.5, 9B parameters**) would be impractical for rapid experimentation. BLIP, with only 247M parameters, **significantly reduces training time and computational cost, allowing us to iterate faster**.

Results obtained with BLIP:

METEOR: 39.8

Improvement wrt best Week 3 Model: +1.2

Improvement wrt best Overall Model: -8.1

* J. Li, D. Li, C. Xiong, and S. Hoi, "BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation," in Proceedings of the 39th International Conference on Machine Learning (ICML), 2022, pp. 12888–12900.

Task c) Explorative Analysis (Definition)

Subsection Motivation: Since our goal is to generate synthetic images that improve the captioning model, we first need to understand where the training data falls short. To do this, we perform a **fine-grained** explorative analysis over the embedding space, which lets us pinpoint underrepresented visual themes and focus our synthetic data generation where it will have the most impact.

Our idea for the Explorative Analysis (Steps Definition)

Step 1: Embedding Extraction + Dimensionality Reduction

- First, we extracted **image embeddings** from both the **training** and **validation** sets using a **pretrained CLIP model**.
- We then fitted a **UMAP** projection on the **training embeddings** to reduce their dimensionality.
- Using the **same fitted UMAP**, we **projected the validation embeddings** into the same reduced space, ensuring a consistent and comparable representation across both splits.



Step 2: Clustering and Assignments

- With the reduced **training** embeddings (2 dimensions), we fit a **clustering algorithm**. Specifically, we use **HDBSCAN (*)** to **automatically detect the number of clusters using its default parameters**.
- Once we have the clusters for the training samples, we **assign the validation samples to the corresponding clusters** using the predict function of the HDBSCAN library.



Step 3: Streamlit Visualization + Cluster Exploration

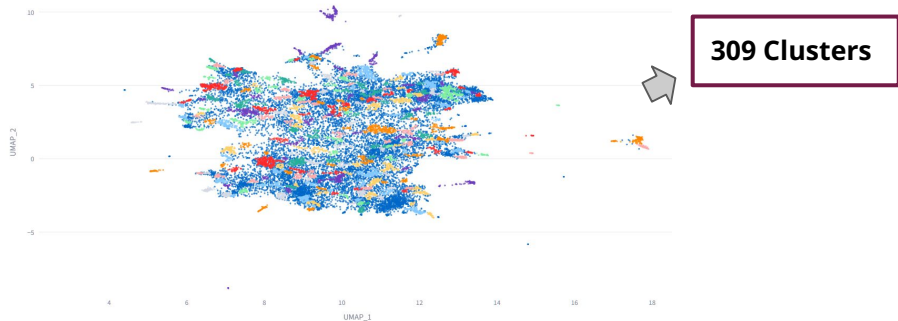
- We designed an interactive visualization using Streamlit **to accelerate the explorative analysis**.
- This tool provides insights into the reduced embedding space and **visualizes the training and validation images for each cluster**.

* C. Malzer and M. Baum, "A Hybrid Approach To Hierarchical Density-based Cluster Selection," in *2020 IEEE International Conference on Multisensor Fusion and Integration for Intelligent Systems (MFI)*, Karlsruhe, Germany, 2020, pp. 223-228, doi: 10.1109/MFI49285.2020.9235263.

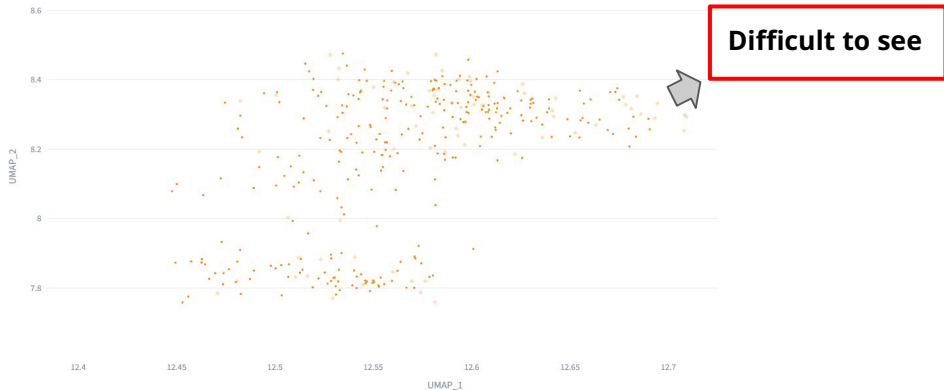
***Note:** Colors are reused due to palette limits.
Spatially separated groups of the same color
are distinct clusters.

Task c) Exploratory Analysis (Visualization)

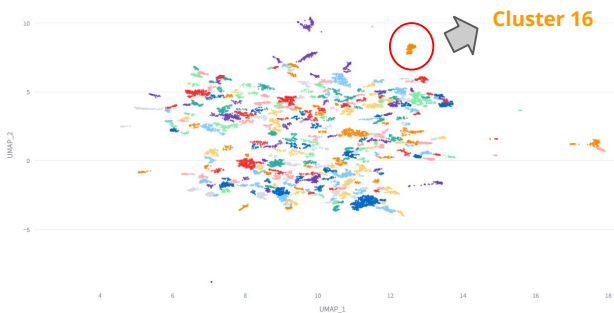
2D Training Samples (with Noise Cluster (-1)) (*)



Cluster 16: training vs validation samples
(points -> train | squares -> validation)



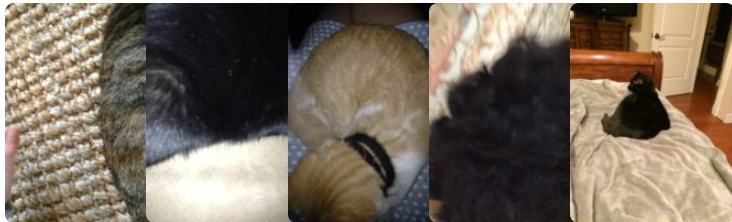
2D Training Samples (without Noise Cluster (-1)) (*)



Cluster 16: Example Samples

Showing 1-20 of 85 images in cluster 16

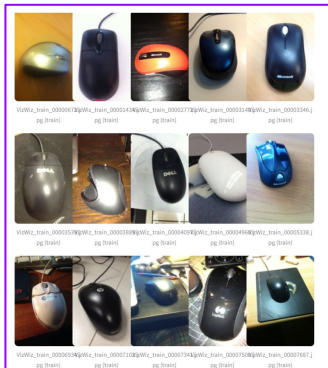
CATS



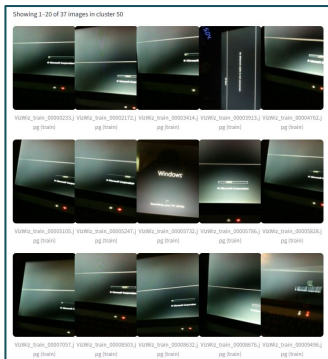
VizWiz_train_0000031.jpg VizWiz_train_0000086.jpg VizWiz_train_0000099.jpg VizWiz_train_0000139.jpg VizWiz_train_00001432.jpg
pg (train) pg (train) pg (train) pg (train) pg (train)

Task c) Exploratory Analysis (Visualization)

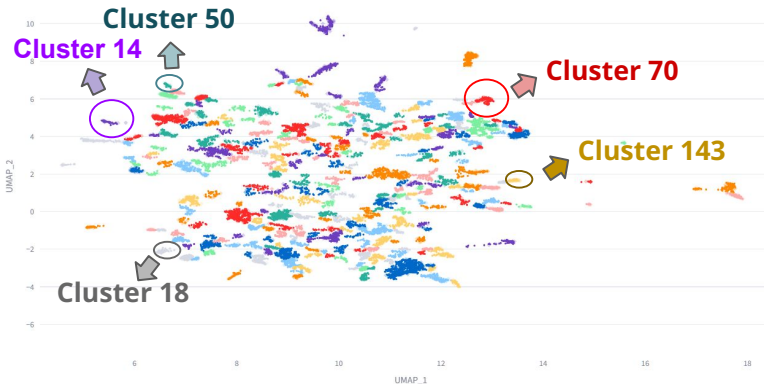
Cluster 14



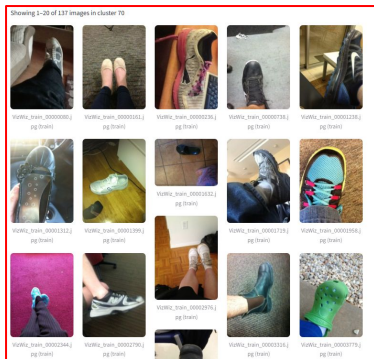
Cluster 50



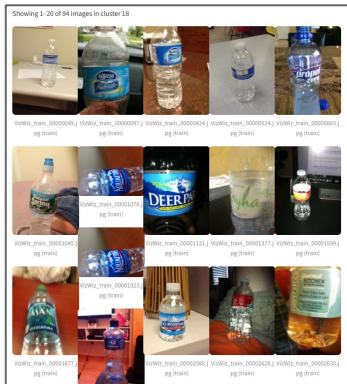
2D Training Samples (without Noise Cluster (-1)) (*)



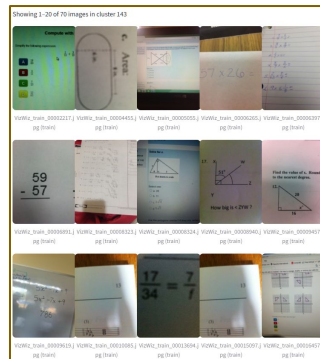
Cluster 70



Cluster 18



Cluster 143



Task c) Quantifying Clusters Performance

Subsection Motivation: With the images now grouped into clusters, we need a way to quantify how each cluster performs. This lets us identify which visual themes the model handles well and where it struggles, pointing us toward the areas that would benefit most from synthetic data augmentation.

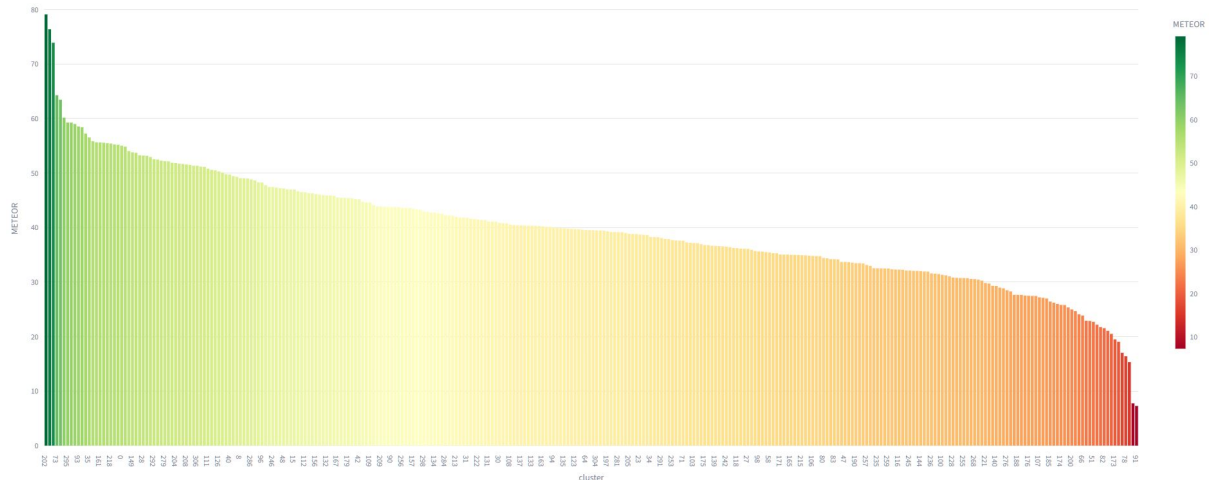
Idea to quantify each cluster performance

- First, we **run our captioning model** from the previous week on the validation samples and **extract the evaluation metrics**.
- Then, we link each validation sample to its assigned cluster and compute the **average METEOR score per cluster**, since METEOR is our primary metric.
- With these per-cluster results in hand, we perform **further exploratory analysis** to identify potential problem areas and guide our synthetic data strategy.

Plot:

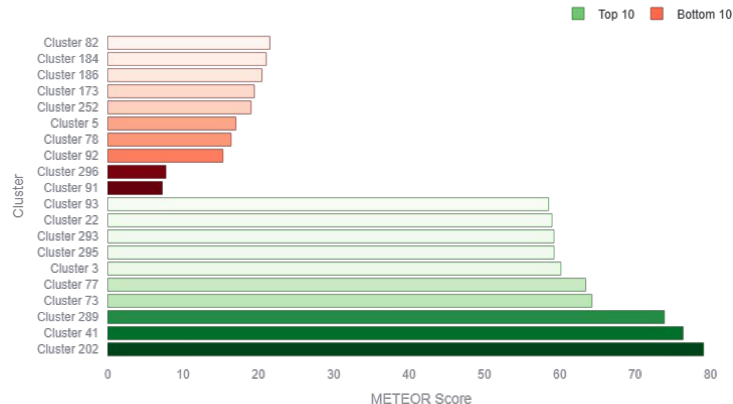
- The plot on the right shows the distribution of average METEOR scores across clusters. In the following slides, **we take a closer look at the most interesting clusters**.

Meteor Score by cluster (sorted) (*)



Task c) Analyzing Clusters Performance

Top 10 Best vs Worst Clusters by METEOR



Key Insights

- The **top 3 clusters** contain training images with **clear semantic**, making them easier for the model to learn and caption accurately. In contrast, the **bottom 3 clusters** contain visually **ambiguous or semantically noisy images**, which are inherently harder to describe.
- This is key when extracting the metrics for the validation. The **semantic quality of the training images appears to be differentiator**.

Top 3 Clusters (Examples)

Cluster 202



Cluster 41

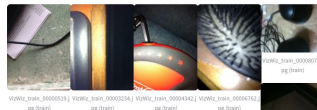


Cluster 289



Bottom 3 Clusters (Examples)

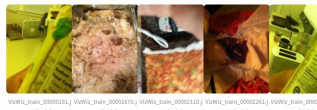
Cluster 91



Cluster 296



Cluster 92

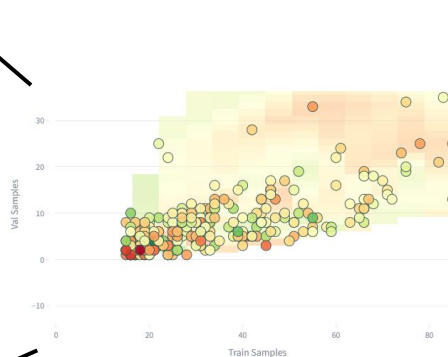


Task c) Analyzing Clusters Performance

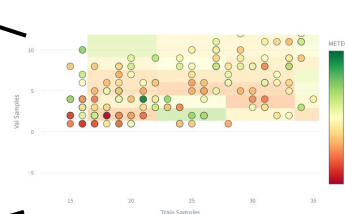
Train vs Val Samples with METEOR (*)



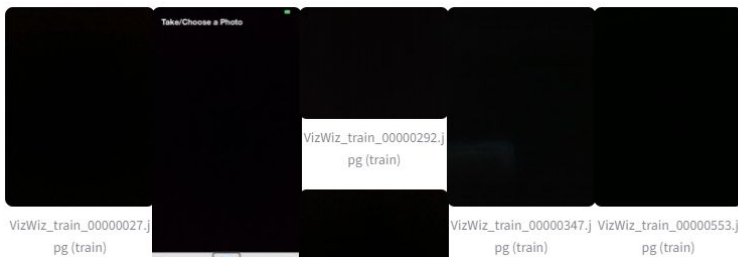
Zoom



Zoom



Cluster 5



Key Insights

- In general, clusters with a high number of train and validation samples **rarely** show low METEOR scores, except in cases like Cluster 5, where the images are inherently difficult (e.g., dark or unclear photos).
- Zooming into clusters with few train and validation samples, we observe a wide spread of METEOR scores, both high and low. Clusters with **high scores** despite limited data **contain semantically clear, easy-to-caption images**, which may **inflate the per-cluster metric**. Conversely, clusters with **low scores** combine two compounding factors: **ambiguous or noisy training images and scarce validation data, making it very likely to obtain poor performance**.

Task c) Our hypothesis and goal

Our reasoning

- Based on our exploratory analysis, it makes sense to **generate synthetic images in some clusters** so that they are visually similar to the cluster's existing training images. **This gives us more control over what we are generating**, rather than adding random or unconstrained images.
- It also makes sense to **prioritize the clusters with the lowest METEOR scores**, aiming to both **improve the global METEOR and equalize performance across clusters**. This way, we are not only improving caption generation overall but **also reducing bias toward certain visual categories**.

Goal: Improve the overall METEOR score of our captioning model by generating targeted synthetic training data for some clusters (*), while reducing the performance gap between the best and worst clusters.

Hypothesis: By generating synthetic images that are semantically consistent with some clusters, the captioning model will learn better representations across all categories, leading to an overall METEOR improvement while closing the performance gap between the strongest and weakest clusters.

Task d) Synthetic Captions and Image Generation

Section Motivation: The goal is to build a cluster-aware pipeline that determines, for each identified cluster, whether synthetic data is needed and how much. When generation is triggered, the pipeline produces captions consistent with the existing training captions in that cluster, which then serve as prompts for Diffusion to generate corresponding images. This ensures augmentation is targeted, not random.

Pipeline (Steps Definon)

Step 1: How many images for each cluster should be generated ?

- As seen in Task c), some clusters already achieve high METEOR scores, so we consider that generating synthetic data for those clusters would not yield significant improvement.
- We computed the **average METEOR score from the validation split**. Clusters scoring above the threshold (39.82) are **assigned 0 synthetic samples**, while clusters below the threshold are **selected for augmentation**.
- For the selected clusters, the number of samples to generate **is inversely proportional to their METEOR score (*)**: worse-performing clusters receive more synthetic samples, following the inverted min-max formula (see formula right *).
- We set the generation boundaries to **min=15** and **max=100** samples per cluster, calibrated so that the total number of synthetic samples across all clusters amounts to approximately 5,000, a number we considered significant relative to the original dataset size.

$$\text{inverted_score} = 1 - \frac{\text{METEOR}_i - \text{METEOR}_{\min}}{\text{METEOR}_{\text{threshold}} - \text{METEOR}_{\min}}$$
$$n_{\text{samples}} = \text{round}(\text{min_samples} + \text{inverted_score} \times (\text{max_samples} - \text{min_samples}))$$

Bottom 5 Clusters (samples to generate)

Cluster ID	Avg METEOR	Samples to Generate
91	7.27	100
296	7.75	99
92	15.31	79
78	16.39	76
5	17.03	75

Task d) Synthetic Captions and Image Generation

Pipeline (Steps Definition)

Step 2: Build a cluster-level prompt to guide synthetic caption generation

- For each cluster, we build a prompt that includes **all or some of its training captions as few-shot examples** to guide the LLM in generating semantically consistent new captions.
- The prompt instructs to generate exactly N captions (as determined in **Step 1**), each with a maximum of 17 words.
- For **large clusters (above-average size)**, we sample only one caption per image **to avoid having too much**. For **small clusters**, we include **all available captions per image to ensure enough context**.
- Since VizWiz is a dataset of real-world photos taken by visually impaired users, many images exhibit low quality (blur, bad lighting, awkward framing, etc.). To reflect this, we instruct to randomly **inject 3-4 visual quality descriptors** ("grainy image", "harsh shadows", "tilted angle"...) into a subset of the generated captions (only 3 out of N), so the synthetic data matches the realistic imperfections of the original dataset.



For each cluster selected to generate images:

Generate **{N}** diverse image captions with maximum 17 words each. Integrate **{K}** of the following visual characteristics/qualities naturally into only 3 of the generated captions: **{quality_1, quality_2, ..., quality_K}**.

Here you have examples of correct captions belonging to this cluster to guide the semantic content of your new captions:

- **{caption_1}**
- **{caption_2}**
- ...
- **{caption_M}**

Task d) Synthetic Captions and Image Generation

Pipeline (Steps Definision)

Step 3: LLM to extract the final prompt for the diffusion model

- We feed each cluster-level prompt from [Step 2](#) into a LLM (**Qwen3.5-2B**) to generate the synthetic captions.
- The output is a CSV where each cluster now has its corresponding generated captions, ready to be used as prompts for the diffusion model in the next step.

However, there are some problems that need to be addressed in a postprocessing steps:

- The LLM output is raw text that may contain numbering, bullet points, or extra formatting . These need to be cleaned and parsed into individual captions.
- The LLM **does not always generate the exact number of captions requested per cluster**. If it **generates more**, we truncate to the target number. If it **generates fewer**, we duplicate existing captions to fill the gap. This is acceptable because the diffusion model's stochastic component ensures that even identical prompts produce visually different images.



Step 4: Generate synthetic images with Diffusion

- Now that we have all the cleaned captions, we use them as prompts for Diffusion. As mentioned in [slide 15](#), we use **two different inference configurations to generate two separate sets of synthetic data**, balancing speed and quality: **250CFG (2 steps, 0 CFG)** and **4S1CFG (4 steps, 1.0 CFG)**. We use this base prompt in order to generate images (*)
- In the following slides, we will **qualitatively compare the generated images** against the real images in each cluster to assess semantic consistency, and ultimately **define a metric to quantify the quality of the augmentation**.

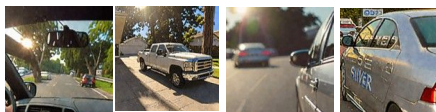
***Note:** Images are shown in generation order: each row compares outputs from the same caption across the two diffusion configurations (250CFG and 451CFG). No relationship with the first row (training samples) they only share the cluster.

Task d) Qualitative Analysis

- * Training samples
- * 250CFG Diffusion Model
- * 451CFG Diffusion Model

27

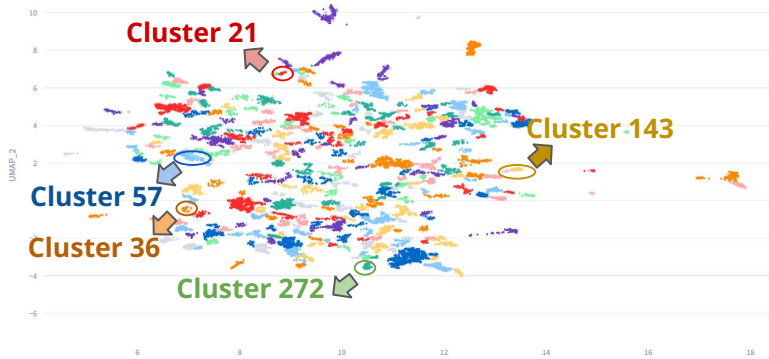
Cluster 21



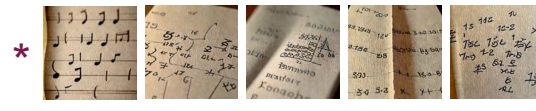
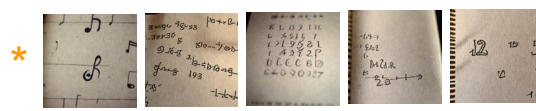
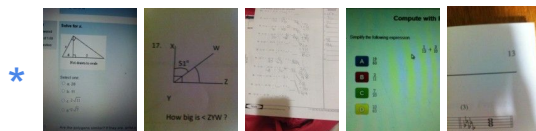
Cluster 57



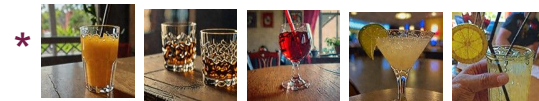
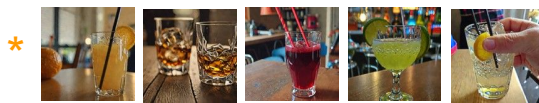
2D Training Samples (without Noise Cluster (-1))



Cluster 143



Cluster 36



Cluster 272



Task d) Quantifying Generated Images

Metric Definition

- After generating the synthetic images, we extracted their **CLIP** embeddings and projected them into the **same UMAP space using the transform fitted on the training data**. Each generated image was then assigned to its nearest cluster (**excluding the noise cluster -1**).
- Since strict **top-1** assignment **may be too restrictive** because nearby clusters often share similar visual semantics we also evaluated assignment accuracy using **top-5, top-10, top-15, and top-20** nearest clusters.
- The **accuracy** for each top-K is defined as the percentage of generated images whose target cluster (the cluster they were generated for) **appears among the K nearest clusters in the UMAP space**.

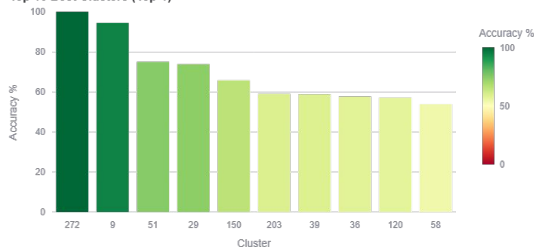
Key Insights

- 2S0CFG slightly outperforms 4S1CFG across all top-K levels, suggesting that fewer diffusion steps with no classifier-free guidance still produces semantically coherent images for this task.
- The cluster with the highest top-1 accuracy is the cereal boxes cluster (as seen in [Slide 27](#))
- When relaxing to top-20, cluster 203 achieves the best accuracy

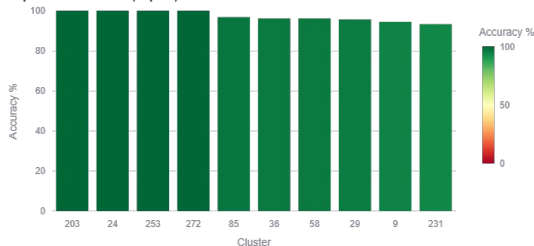
Model ID	Top 1	Top 5	Top 10	Top 15	Top 20
2S0CFG	10.3	25.8	35.4	41.2	45.9
4S1CFG	10.2	24.9	35.5	40.3	44.4

2S0CFG

Top 10 Best Clusters (Top-1)

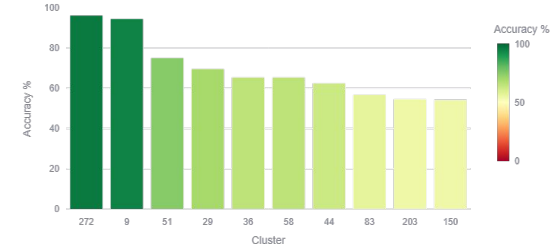


Top 10 Best Clusters (Top-20)

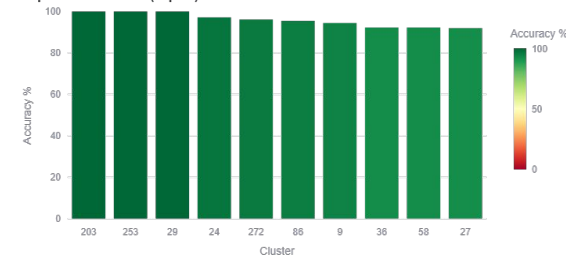


4S1CFG

Top 10 Best Clusters (Top-1)



Top 10 Best Clusters (Top-20)



Task e) Experiment Setup & Strategies

Model Selection

- We selected our **fully fine-tuned BLIP (*) model** (previously trained on the full VizWiz dataset) as our primary baseline.
- **Previous Week Baselines:**
 - Pretrained BLIP METEOR: **32.3**
 - Fine-tuned BLIP (VizWiz only) METEOR: **39.8**
- To explore the impact of synthetic data, **we evaluated 6 different strategies** grouped into two main approaches

Training Strategies

1. **Data Augmentation (Mixed):** Starting from the **pretrained BLIP base**, we trained on the original VizWiz dataset mixed with different synthetic subsets:
 - *VizWiz + 2S0CFG*
 - *VizWiz + 4S1CFG*
 - *VizWiz + Combined Synthetic Data (Both)*
2. **Fine-Tuning on Synthetic (Isolated):** Starting from our **previously fine-tuned BLIP**, we performed further training exclusively on synthetic data to measure distribution shifts:
 - *2S0CFG Only*
 - *4S1CFG Only*
 - *Combined Synthetic Data Only (Both)*

Default Hyperparameters

Parameter	Value
Batch Size	16
Epochs	10 (Best model checkpoint saved per epoch)
Optimizer	AdamW
Learning Rate (Pretrained)	2e-5 (Standard fine-tuning for base models)
Learning Rate (Fine-tuned)	5e-6 (Stable refinement for already optimized models)

Note: The validation set remains untouched (consisting exclusively of original VizWiz data). We solely expanded the training set as a data augmentation technique.

* J. Li, D. Li, C. Xiong, and S. Hoi, "BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation," in Proceedings of the 39th International Conference on Machine Learning (ICML), 2022, pp. 12888–12900.

Task e) Results

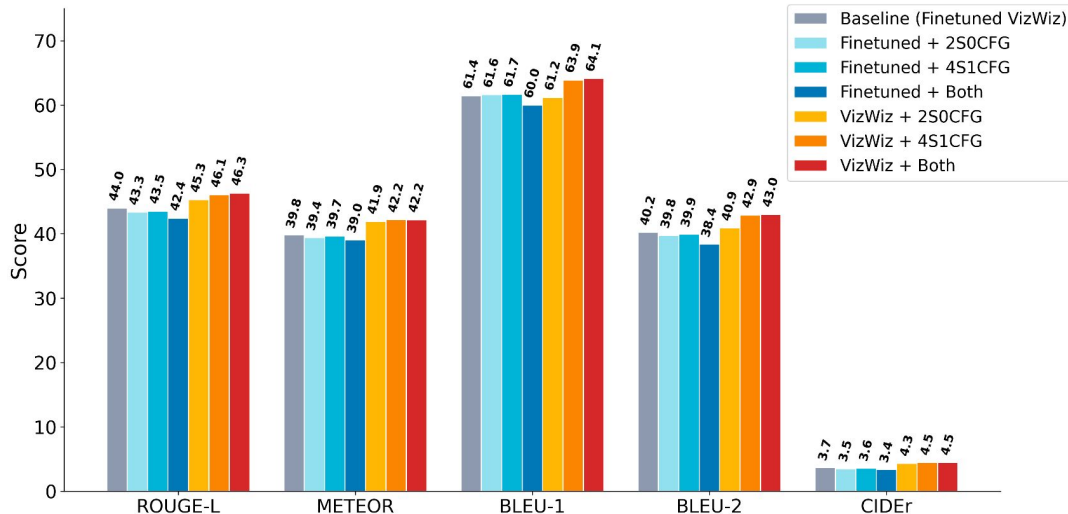
Results & Key Insights

- **The Power of Mixed Augmentation:** Taking the pretrained BLIP and fine-tuning it on a **mixed dataset** (Original VizWiz + Synthetic) **consistently outperforms** the baseline across all metrics.
- **The "Isolated Training" Trap:** Fine-tuning the already adapted model **exclusively on synthetic data** actually **degrades performance** (*METEOR drops from 39.8 to ~39.4*). **Why?** This causes a **covariate shift** and **catastrophic forgetting**. The synthetic images lack the unique noisy/blurry artifacts of real VizWiz photos. Training solely on synthetic data makes the model **"forget"** the real-world target distribution it needs for the validation set.
- **Quality vs. Quantity** (CFG & Steps impact): Within the mixed strategies, the 4S1CFG subset shows a **subtle improvement** over 2S0CFG (42.2 vs 41.9). **Why?** This slight edge is likely due to the combined effect of introducing Classifier-Free Guidance (CFG) and **doubling the denoising steps (4 vs. 2)**. Together, these generate images with slightly tighter image-text alignment, providing a marginally cleaner learning signal.
- **Maximum Synergy ("Both"):** While 4S1CFG and the combined dataset ("Both") achieve the exact **same top METEOR score** (42.2), combining both datasets **maximizes visual diversity**. This synergy pushes the **secondary metrics** to their absolute highest, making it our definitive final choice.

Decision

- **Final Model:** VizWiz + Both (Mixed Augmentation)
- **METEOR: 42.2**
- **Improvement on pretrained BLIP: +9.9**
- **Improvement on finetuned BLIP (VizWiz only): +2.4**

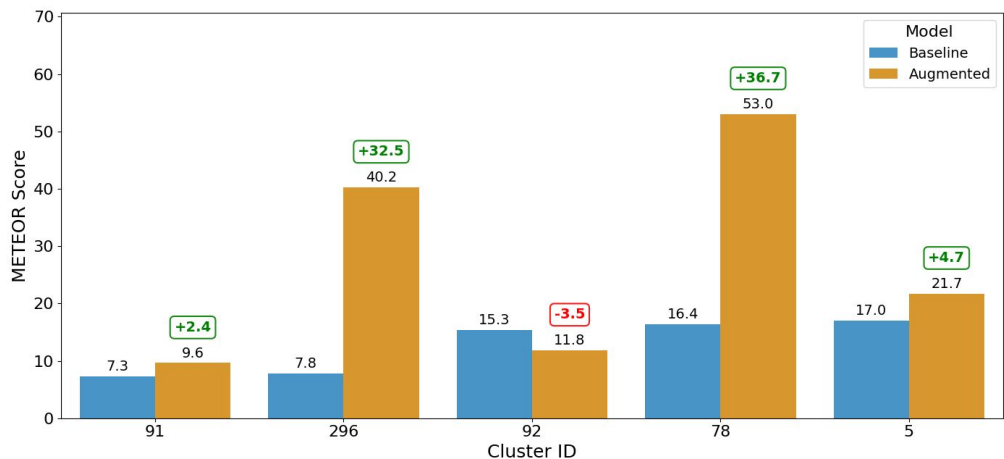
BLIP Finetuned Models Comparison



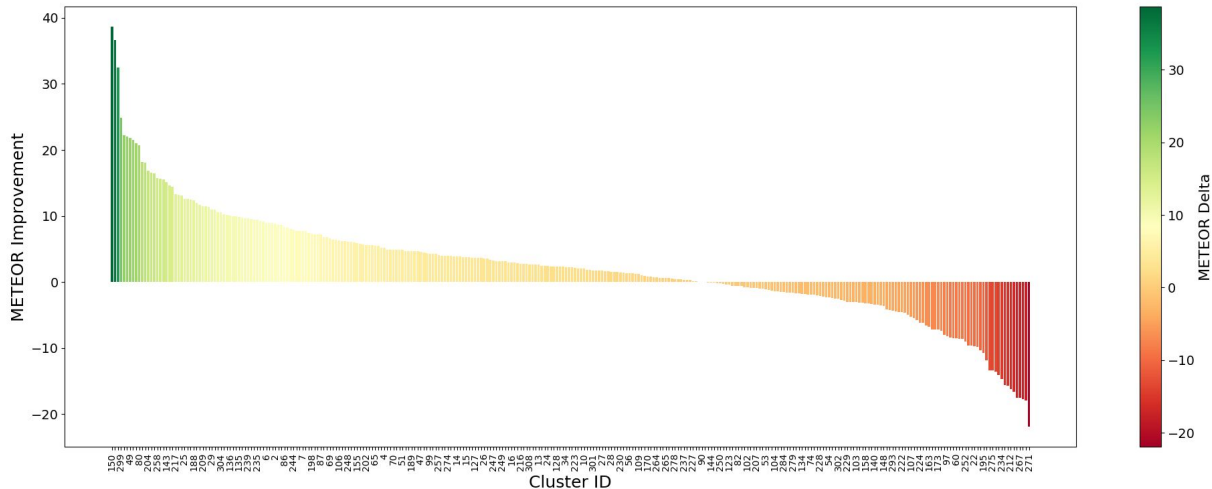
Task e) Analysis

- **Primary Objective:** Evaluate if injecting targeted synthetic data improves the captioning model's performance on its **worst-performing clusters**.
- **Targeted Success (Top Plot):** Looking at the top 5 weakest baseline clusters, **4 out of 5 significantly improved** their METEOR scores. This directly validates our core hypothesis: synthesizing data for the model's specific "blind spots" actually works.
- **Overall Impact (Bottom Plot):** Examining the METEOR delta across all clusters shows that **64.59% (~2/3) of the dataset improved**. Furthermore, the magnitude of the positive gains heavily outweighs the negative drops.
- **Understanding the Trade-offs:** While most clusters improved, a minority worsened. This is a known trade-off likely caused by **generative noise** or an introduced **domain shift**, where the synthetic images slightly distorted the original data distribution.
- **Final Verdict: YES, the goal was achieved.** Despite minor noise, the overall impact of synthetic data augmentation is overwhelmingly positive, proving it is a highly viable method for patching dataset biases.

Top 5 Worst Performing Clusters: Baseline vs Augmented



METEOR Delta per Cluster (Augmented - Baseline)



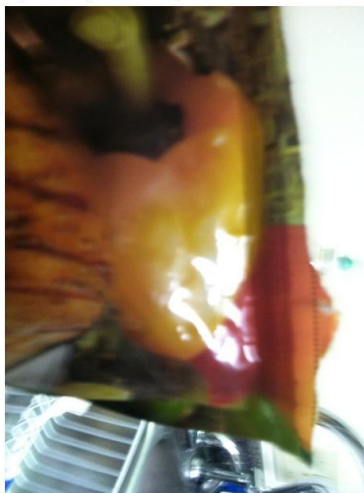
Task e) Qualitatives

Here we examine examples from our weakest clusters to see how synthetic data altered model behavior.

- **Cluster 92 (Ambiguity Limit):** The image is heavily obscured, yet both models logically guess "a bag of food". This highlights **Ground Truth subjectivity**: augmentation cannot fix fundamentally unresolvable images, and the model defaults to a safe guess.
- **Cluster 78 (Curing Hallucinations):** The baseline hallucinates a "computer screen," while the augmented model correctly identifies a "cell phone". Synthetic data successfully **recalibrated object recognition** to fix this blind spot.
- **Cluster 296 (Breaking Repetition):** The baseline gets stuck in a classic LLM repetitive loop ("darigo brand..."). The augmented model breaks this loop to output a coherent description, proving synthetic data helps **regularize the language decoder** against noisy visual features.

Key Takeaway: The improvements are not just statistical noise. Targeted synthetic data actively **cures specific model pathologies** (hallucinations and decoding loops) to build a more robust captioner.

Cluster 92



GT Caption

A blurry close up of what appears to be a bag of cany obscures a chrome-like utensil that looks like the grill on a fany car.

Caption - Baseline

a bag of food is on top of a table.

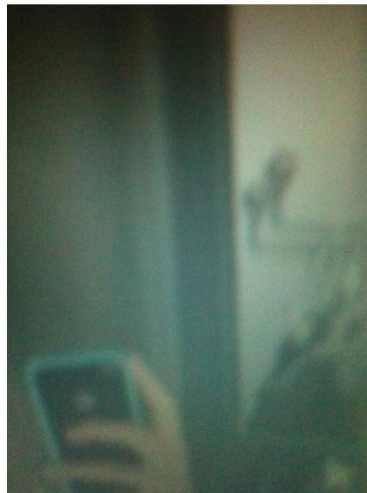
METEOR: 15.75

Caption - Augmented

a bag of food is on top of a table.

METEOR: 15.75

Cluster 78



GT Caption

a hand holding a smartphone with a blue edged case sitting in a living room

Caption - Baseline

a close up of a computer screen with a white and green image.

METEOR: 16.39

Caption - Augmented

a person is holding a cell phone in their hand.

METEOR: 53.05

Cluster 296

32



GT Captions

- Part of a blue cartoon of Darigo Milk with a white cap
- In this picture is a image on top of a carton

Caption - Baseline

a box of darigo brand darigo brand darigo brand darigo brand darigo brand

METEOR: 7.75

Caption - Augmented

a package of unidentifiable product with a white lid.

METEOR: 40.24

Conclusions & Future Work

Conclusions

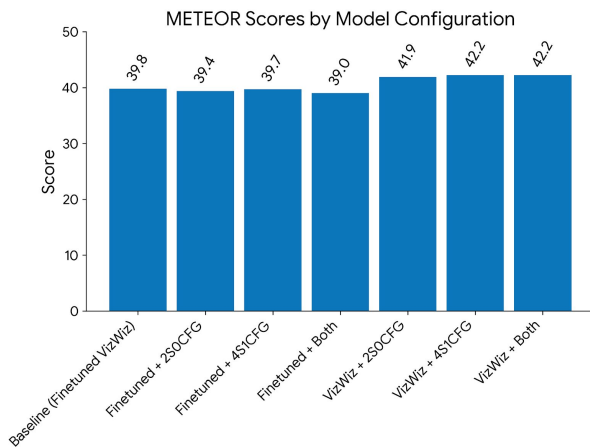
- **Targeted Augmentation is Effective:** Injecting targeted synthetic data successfully improved our overall captioning model, increasing the global METEOR score by 2.4 points and validating our primary hypothesis.
- **High Impact from Minimal Data:** This global improvement is especially significant because we only augmented a tiny fraction of the dataset. We excluded Cluster -1 (unclassified noise, representing 61.2% of data) and ignored above-average clusters, meaning this gain was driven entirely by fixing the bottom 27.3% of the data.
- **Understanding Degradation:** While the net impact was highly positive, some clusters worsened. This is likely due to **domain shift** (synthetic images looking too "clean" compared to the messy VizWiz dataset) or **semantic mismatch** (the generated prompts failing to capture the specific visual ambiguity of those clusters).

Future Work

- **Density-Aware Sampling:** Instead of purely using average METEOR, factor in the **number of samples per cluster** to determine exactly how many synthetic images each weakness requires.
- **Clustering Optimization:** Tune the clustering hyperparameters (e.g., HDBSCAN) to aggressively reduce the size of Cluster -1. Unlocking that remaining 61.2% of data will reveal new sub-clusters we can target.
- **Data Scaling Laws:** Generate a much larger volume of synthetic images to test the saturation point and finding exactly where more images stop improving the METEOR score.
- **Alternative Diffusion Applications:** Move beyond text-to-image generation. Use Image-to-Image (Img2Img) or specialized diffusion models to directly **restore, deblur, or enhance** the existing poor-quality images in the dataset before passing them to the captioner.
- **Pipeline Upgrades:** Refine the LLM prompt engineering for better synthetic captions, and integrate state-of-the-art models. (We briefly tested SD 3.5 with incredible results, but excluded it due to the high computational costs. Fully integrating this is a clear next step).

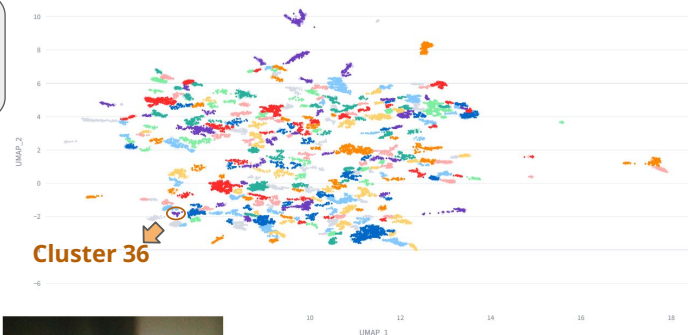


Summary

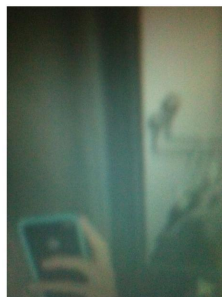


- ★ Training samples
- ★ 250CFG Diffusion Model
- ★ 451CFG Diffusion Model

2D Training Samples (without Noise Cluster (-1))



Cluster 36



GT Caption

a hand holding a smartphone
with a blue edged case sitting in a
living room



Caption - Baseline

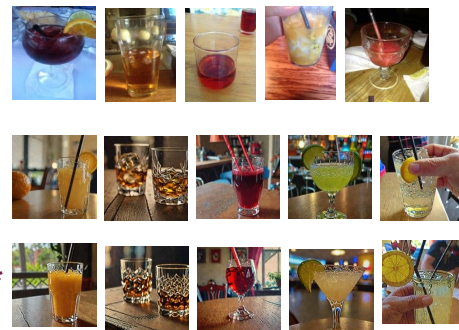
a close up of a computer screen
with a white and green image.

METEOR: 16.39

Caption - Augmented

a person is holding a cell phone
in their hand.

METEOR: 53.05



Conclusions

- **IDEA:** Extract embeddings using **CLIP**, reduction of dimensionality with **UMAP** and a clustering algorithm (**HDBSCAN**) to cluster images, and then for those clusters with **lower METEOR** generate images of that cluster to have more training data.
- **Step 1:** How many images for each cluster should be generated?
- **Step 2:** Build a cluster-level prompt to guide synthetic caption generation?
- **Step 3:** LLM to extract the final prompt for the diffusion model
- **Step 4:** Generate synthetic images with Diffusion
- **Step 5:** Retrain the model using 6 strategies

Results, conclusions and future work

- **+2.4** of METEOR improvement **taking only into account 27.3 % of training data.**
- As a **future work:** **Density-Aware Sampling, Clustering Optimization, Data Scaling Laws, Alternative Diffusion Applications, Pipeline Upgrades**